

Good catch — this line is from the **SMART on FHIR / OAuth2 best practices** for **client authentication**.
Let's break it down step by step:

1. Client authentication in OAuth2

When your app calls the **token endpoint** (to exchange an authorization code, or to get a `client_credentials` token), Epic (or any OAuth server) needs to know:

"Who is this app?"

This is **client authentication**. There are different ways apps prove their identity:

- **Client Secret** (symmetric)
 - Server and client share a secret string.
 - App sends it along with the token request (`client_id` + `client_secret`).
 - Problem: if you put the secret into a mobile/web app, users can extract it → insecure.
 - Safe only when kept **server-side**.
 - **Private Key JWT** (asymmetric, recommended)
 - Instead of a shared secret, the app generates a **JWT signed with its private key**.
 - The authorization server (Epic) verifies it using the **public key** you uploaded when registering the app.
 - No secret is stored on the client; only the private key stays with your server.
-

2. What `private_key_jwt` looks like

When your app requests a token, it includes:

- `client_id` = your app's ID
- `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer`
- `client_assertion=<signed JWT>`

The JWT claims (RFC 7523):

- `iss` : your `client_id`
- `sub` : your `client_id`

- `aud` : Epic's **token endpoint URL**
- `exp` : expiration (5 min max usually)
- `jti` : unique ID (to prevent replay)

The JWT is signed with your **private key**.

Epic verifies it using the **public key** you registered.

3. Why “prefer asymmetric client authentication”

- **More secure**: No long-lived shared secret.
 - **No risk if Epic is compromised**: Epic only stores your public key.
 - **Server-side only**: The private key stays on your backend, never in mobile apps or browsers.
 - **Required for Backend Services (SMART system-level apps)** – Epic explicitly supports `private_key_jwt` there.
-

4. In practice with Epic

- **Backend apps (system-level, client_credentials)**: you must use `private_key_jwt`.
 - **Interactive apps (auth code + PKCE)**: some Epic sites may allow `client_secret`, but best practice is still `private_key_jwt` if they support it.
 - You configure your **JWK Set (public keys)** in Epic App Orchard (or the customer's Epic App registry).
-

So, the sentence means:

When you build a confidential app (server-based, not a public JS/mobile app), **don't rely on client secrets**. Instead, use **asymmetric authentication (`private_key_jwt`)**. That way, only your server holds the private key, and Epic verifies it with your registered public key.

Do you want me to show you an **actual JWT example** (with claims + RS256 signature) like the one Epic expects at the token endpoint?